Métodos Cuantitativos

Clase 13 - Introducción a la regresión logística

Unidad III: Regresión Logística

Repaso

- Recordemos que la regresión lineal es una técnica de modelado predictivo que se utiliza para **predecir un valor continuo** basado en una o más variables independientes. La fórmula básica de la regresión lineal es:
- $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \varepsilon$
- Donde y es la variable dependiente, x_i son las variables independientes, β_i son los coeficientes del modelo y ε es el término de error.

Repaso

- Para el siguiente ejemplo utilizaremos el dataset del *titanic*, donde la variable `survived` es de tipo categórica, indicando con un `0` si la persona falleció y con un `1` si sobrevivió.
- Intentaremos ajustar un modelo de regresión lineal usando `survived` como variable dependiente.

Repaso

```python
import seaborn as sns
import numpy as np
import statsmodels.api as sm
from statsmodels.tools.eval_measures import rmse

# cargar datos
data = sns.load_dataset('titanic')
data.drop(columns=["deck"], inplace=True)
data.dropna(inplace=True)
data["female"] = np.where(data["sex"] == "female", 1, 0)

# ajustar modelo de regresión lineal
X = data[['age', 'fare', 'pclass', 'female']]
y = data['survived']

X = sm.add_constant(X)

modelo = sm.OLS(y, X)
```

```
R2: 0.38800107696311115
RMSE: 0.38394985462506137
```

¿Cómo podemos interpretar el RMSE?

Repaso

```python
# observemos los valores ajustados
print(resultados.fittedvalues[:10])
```

```
0     0.118756
1     0.913165
2     0.574369
3     0.928368
4     0.047646
6     0.346813
7     0.229165
8     0.569110
9     0.841662
10    0.695388
dtype: float64
```

Dado que la variable dependiente es categórica binaria (0 y 1),

¿qué interpretación podemos extraer de los resultados?

Repaso

```
# y los rangos mínimos y máximos
print(min(resultados.fittedvalues), max(resultados.fittedvalues))
```

```
-0.16586215656746328 1.115612553161375
```

¿Cómo podemos interpretar los valores mínimos y máximos predichos por el modelo?

¿Qué veremos hoy?

- **Introducción a clasificación**
- Introducción a la regresión logística
- Regresión logística en Python
- Odds y log-odds
- Evaluación del modelo logístico

Introducción a Clasificación

Introducción a Clasificación

- Cuando intentamos usar una regresión lineal para resolver un problema de clasificación binaria, nos encontramos con un gran inconveniente: las predicciones del modelo no están restringidas al rango [0, 1], lo que las hace poco interpretables como probabilidades.
- Esto nos lleva a buscar un enfoque más adecuado para problemas de clasificación: la regresión logística.

Introducción a Clasificación

¿Qué es la clasificación?

- La clasificación es una tarea de *machine learning* donde el objetivo es asignar una etiqueta o categoría a una observación. En el caso de **clasificación binaria**, queremos predecir una de dos clases, por ejemplo:
  - 0: No ocurre el evento (Clase negativa).
  - 1: Ocurre el evento (Clase positiva).
- El modelo busca aprender una frontera de decisión que permita separar las observaciones de cada clase.

Introducción a Clasificación

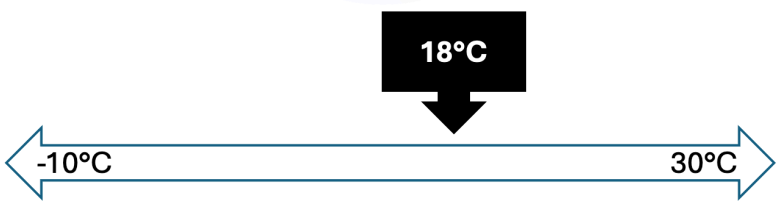
- Diferencias clave entre regresión y clasificación
  - En **regresión**, el objetivo es predecir valores continuos, como precios o temperaturas.
  - En **clasificación**, el objetivo es predecir una categoría o clase discreta, como “Aprobado” o “Rechazado”.

Introducción a Clasificación

Supongamos que queremos construir un modelo meteorológico. Las diferencias entre los enfoques son fundamentales para determinar la forma en que se abordará el problema:

Regresión

¿Qué temperatura habrá mañana?



Predicción

$\hat{y} = 18$

Clasificación

¿Hará calor o frio mañana?

Predicción

$\hat{y} = \text{“}Calor\text{”}$

¿Qué veremos hoy?

- Introducción a clasificación
- **Introducción a la regresión logística**
- Regresión logística en Python
- Odds y log-odds
- Evaluación del modelo logístico

Introducción a la Regresión Logística

Introducción a la Regresión Logística

- La regresión logística se utiliza para **modelar la probabilidad de que una observación pertenezca a una de dos clases posibles**.
- La función logística, utiliza la **función sigmoide** para convertir la salida de la combinación lineal en una probabilidad:
- $$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Introducción a la Regresión Logística

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Donde z es el valor que se obtiene a partir de una combinación lineal de las características (variables independientes) del modelo:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

En esta fórmula:

- $x_1, x_2, \ldots, x_n$ son las variables independientes (características) del modelo.
- $\beta_0, \beta_1, \ldots, \beta_n$ son los coeficientes o pesos que el modelo ajusta durante el entrenamiento.
- z es el valor que luego pasa a la función sigmoide, que lo convierte en una probabilidad entre 0 y 1.

Función sigmoide

- Supongamos que tenemos un modelo de regresión logística con una única variable X:
- La función z sería: $z = \beta_0 + \beta_1 X$
- Aplicando la función sigmoide:
  - $P(y = 1|X) = \frac{1}{1+e^{-\beta_0+\beta_1 X}}$
- Esto nos da la **probabilidad** de que $y = 1$ para un valor dado de X.

Introducción a la Regresión Logística

¿Qué significa la salida de la Regresión Logística?

Introducción a la Regresión Logística

Clasificación a partir de probabilidades

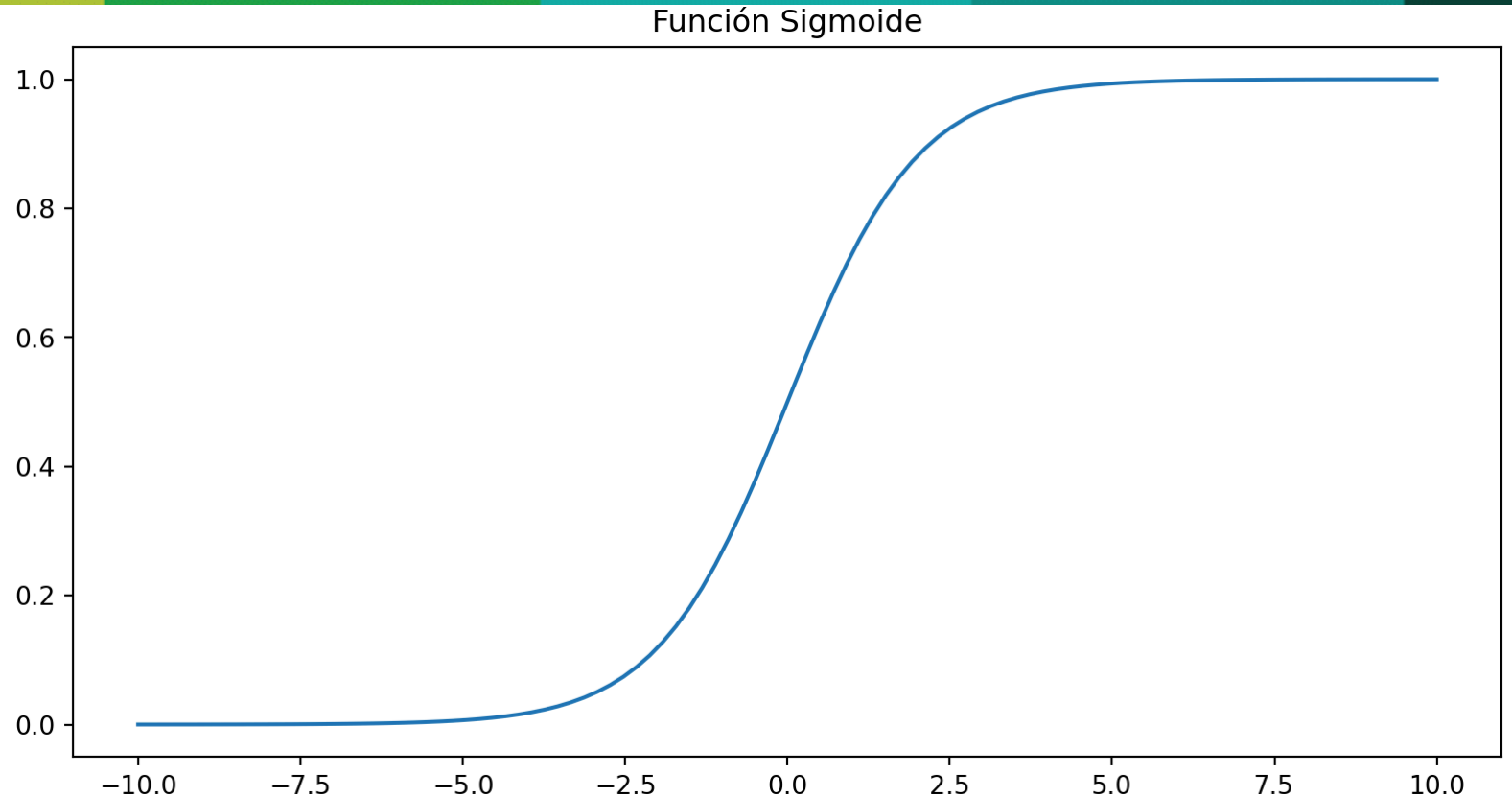
- Una vez calculada la probabilidad, se asigna una clase usando un umbral (generalmente 0.5):
  - Si $P(y = 1|X) \geq 0.5$, se clasifica como 1.
  - Si $P(y = 1|X) < 0.5$, se clasifica como 0.

Introducción a la Regresión Logística

Función sigmoide

La función sigmoide se representa de la siguiente manera, donde el eje y indica la probabilidad de pertenecer a la clase modelada:

```python
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

z = np.linspace(-10, 10, 100)
plt.plot(z, sigmoid(z))
plt.title("Función Sigmoide")
plt.show()
```


Función Sigmoide
1.0
0.8
0.6
0.4
0.2
0.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

¿Qué veremos hoy?

- Introducción a clasificación
- Introducción a la regresión logística
- **Regresión logística en Python**
- Odds y log-odds
- Evaluación del modelo logístico

Regresión Logística en Python

Regresión Logística en Python

- También usaremos la librería `statsmodels` para modelar la regresión logística, pero en este caso usaremos el módulo `Logit`.

Regresión Logística en Python

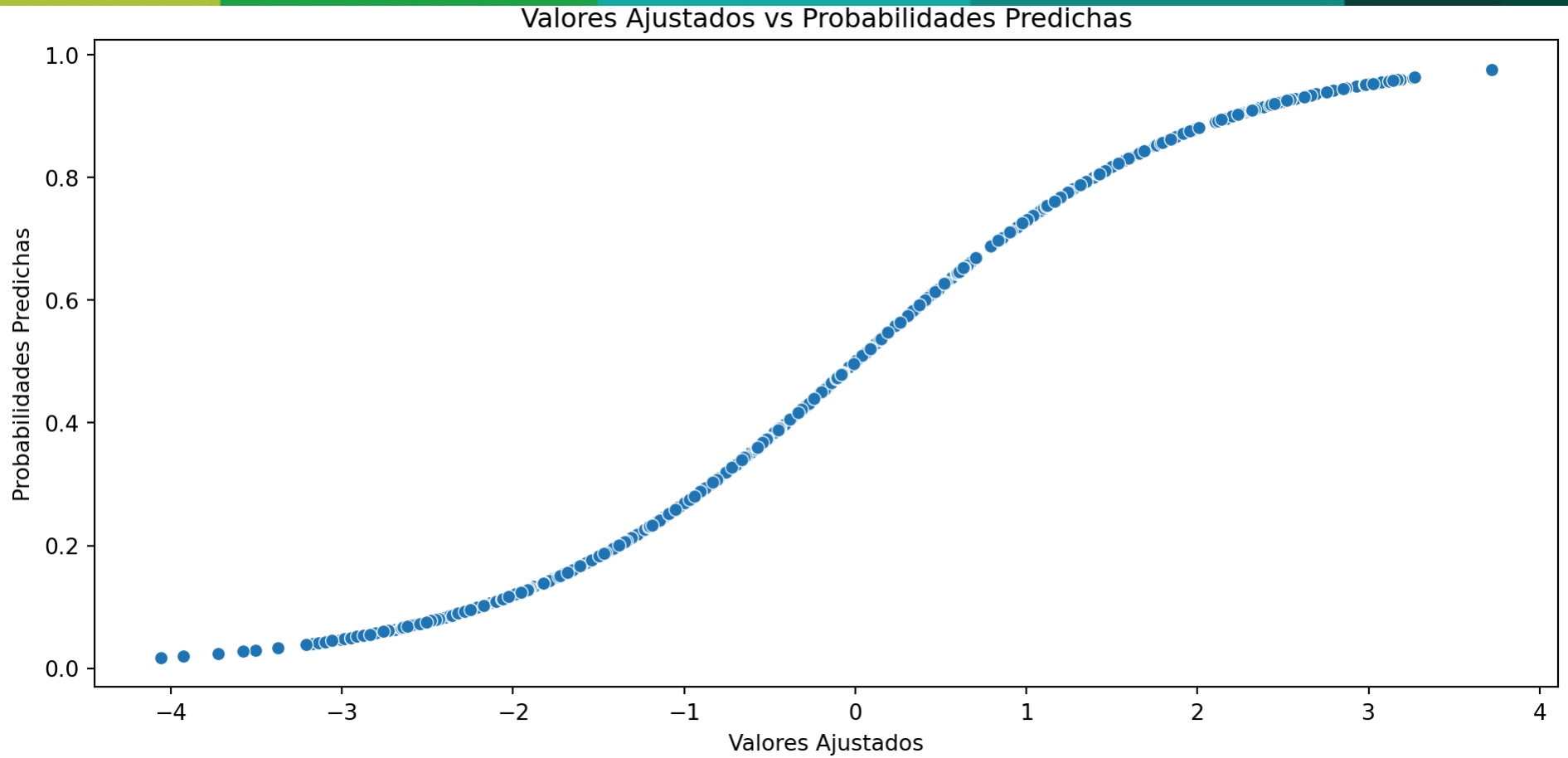
```python
# ajustar modelo de regresión logística
X = data[['age', 'fare', 'pclass', 'female']]
y = data['survived']

X = sm.add_constant(X)

modelo = sm.Logit(y, X)
resultados = modelo.fit(disp=0)

print(resultados.predict(X))
```

```
0      0.105514
1      0.912751
2      0.555954
3      0.920488
4      0.068043
         ...
885    0.439224
886    0.258735
887    0.953818
889    0.564584
890    0.075404
Length: 712, dtype: float64
```

Regresión Logística en Python

Si graficamos el resultado del modelo, podemos observar la forma Sigmoide:

```
import seaborn as sns
sns.scatterplot(x=resultados.fittedvalues, y=resultados.predict(X), color='
plt.title("Valores Ajustados vs Probabilidades Predichas")
plt.xlabel("Valores Ajustados")
plt.ylabel("Probabilidades Predichas")
plt.tight_layout()
plt.show()
```


Valores Ajustados vs Probabilidades Predichas
Probabilidades Predichas
Valores Ajustados
1.0
0.8
0.6
0.4
0.2
0.0
−4
−3
−2
−1
0
1
2
3
4

Regresión Logística en Python

```python
# agregar predicción
data["probabilidad_survived"] = resultados.predict(X)

# imprimir resultados
print(data[['age', 'fare', 'pclass', 'female', 'survived', 'probabilidad_su
```

```
      age     fare  pclass  female  survived  probabilidad_survived
0    22.0   7.2500       3       0         0               0.105514
1    38.0  71.2833       1       1         1               0.912751
2    26.0   7.9250       3       1         1               0.555954
3    35.0  53.1000       1       1         1               0.920488
4    35.0   8.0500       3       0         0               0.068043
..    ...      ...     ...     ...       ...                    ...
885  39.0  29.1250       3       1         0               0.439224
886  27.0  13.0000       2       0         0               0.258735
887  19.0  30.0000       1       1         1               0.953818
889  26.0  30.0000       1       0         1               0.564584
890  32.0   7.7500       3       0         0               0.075404

[712 rows x 6 columns]
```

Regresión Logística en Python

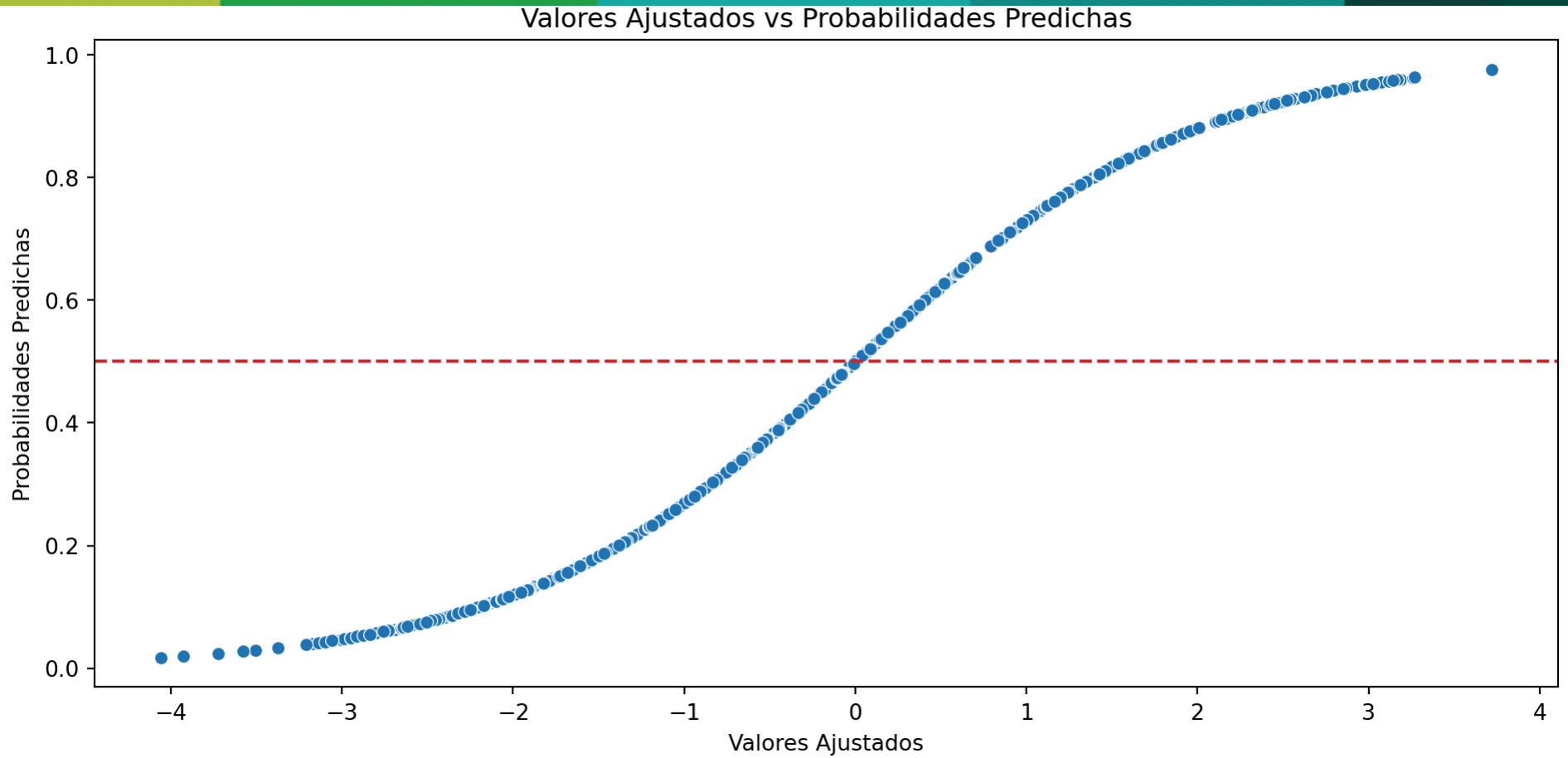
```python
import numpy as np
# clasificar predicción
data["prediccion"] = np.where(data["probabilidad_survived"] > 0.5, 1, 0)

# imprimir resultados
print(data[['survived', 'probabilidad_survived', 'prediccion']])
```

```
     survived  probabilidad_survived  prediccion
0           0               0.105514           0
1           1               0.912751           1
2           1               0.555954           1
3           1               0.920488           1
4           0               0.068043           0
..        ...                    ...         ...
885         0               0.439224           0
886         0               0.258735           0
887         1               0.953818           1
889         1               0.564584           1
890         0               0.075404           0

[712 rows x 3 columns]
```

Regresión Logística en Python

La línea roja indica el “punto de corte” de la probabilidad:

```
import matplotlib.pyplot as plt
sns.scatterplot(x=resultados.fittedvalues, y=resultados.predict(X), color='
plt.axhline(0.5, color='tab:red', linestyle='--')
plt.title("Valores Ajustados vs Probabilidades Predichas")
plt.xlabel("Valores Ajustados")
plt.ylabel("Probabilidades Predichas")
plt.tight_layout()
plt.show()
```


Valores Ajustados vs Probabilidades Predichas
Probabilidades Predichas
Valores Ajustados
1.0
0.8
0.6
0.4
0.2
0.0
−4
−3
−2
−1
0
1
2
3
4

¿Qué veremos hoy?

- Introducción a clasificación
- Introducción a la regresión logística
- Regresión logística en Python
- **Odds y log-odds**
- Evaluación del modelo logístico

Odds y Log-Odds

Odds y Log-Odds

- En el contexto de la regresión logística y otros modelos de clasificación, es común encontrar términos como **odds y log-odds**.
- Estos conceptos son fundamentales para entender cómo se interpretan las probabilidades y los coeficientes en modelos como la regresión logística.

Odds

- Los **odds** se refieren a la relación de la probabilidad de que ocurra un evento con respecto a la probabilidad de que no ocurra. Matemáticamente, los odds se calculan como:
- $\text{Odds} = \frac{P}{1-P}$
- Donde P es la probabilidad del evento. Los odds son una forma de expresar la probabilidad que es particularmente útil para comparar la probabilidad relativa de un evento en diferentes condiciones.

Odds

- Ejemplo: Si un evento tiene una probabilidad de 0.8, los odds serían:
- $\text{Odds} = \frac{0.8}{0.2} = 4$
- Esto significa que el evento es cuatro veces más probable que ocurra que no ocurra.

Odds

Analicemos el siguiente ejemplo:

```
import pandas as pd
from sklearn.model_selection import train_test_split

# generamos la matriz de atributos
X = data[['age', 'fare', 'pclass', 'female']]
# añadimos el intercepto a la matriz atributos
X = sm.add_constant(X)
# generamos el vector respuesta
y = data['survived']

# separamos los datos en entrenamiento y validación
X_train, X_test, y_train, y_test = train_test_split(
  X, y, random_state=42, test_size=0.2
)

# ajustamos el modelo con los datos de entrenamiento
modelo = sm.Logit(y_train, X_train)
resultados = modelo.fit(disp=0)
```

Odds

```python
# asignamos los odds -código sólo útil para este ejemplo-
X_test["y_prob"] = y_pred_prob
X_test["odds"] = odds

# reportamos el resultado
print(X_test.head(5))
```

```
     const   age      fare  pclass  female    y_prob       odds
641    1.0  24.0   69.3000       1       1  0.949288  18.719333
496    1.0  54.0   78.2667       1       1  0.874002   6.936641
262    1.0  52.0   79.6500       1       0  0.338803   0.512409
311    1.0  18.0  262.3750       1       1  0.927696  12.830556
551    1.0  27.0   26.0000       2       0  0.240611   0.316848
```

En este caso, podemos interpretar que el primer registro tiene **18.7 veces más probabilidades de sobrevivir** en comparación con el último, el cual tiene apenas **0.3 veces las probabilidades de sobrevivir**.

Log-Odds

- Los **log-odds** corresponden el logaritmo natural de los odds.
- Este concepto es muy utilizado en regresión logística porque transforma los odds, que pueden variar entre 0 e infinito, en valores que van de $-\infty$ a ∞.
- Esto es útil porque la regresión lineal (que subyace a la regresión logística) asume que las variables independientes están linealmente relacionadas con la variable dependiente en esta escala.

Log-Odds

$$\text{log-odds} = \log\left(\frac{P}{1-P}\right)$$

```python
# aplicamos logaritmo natural a los odds
log_odds = np.log(odds)

# reportamos los primeros 10 resultados
print(log_odds[:10])
```

```
641    2.929557
496    1.936818
262   -0.668633
311    2.551830
551   -1.149334
550    0.366262
279   -0.168758
268    1.585615
110   -0.425828
554    0.286830
dtype: float64
```

Log-Odds

Y cuando aplicamos la función *sigmoide* a los *log-odds*, obtenemos las probabilidades estimadas por el modelo logístico.

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

print(sigmoid(log_odds[:5]))
```

```
641    0.949288
496    0.874002
262    0.338803
311    0.927696
551    0.240611
dtype: float64
```

```
print(y_pred_prob[:5])
```

```
641    0.949288
496    0.874002
262    0.338803
311    0.927696
551    0.240611
dtype: float64
```

¿Qué veremos hoy?

- Introducción a clasificación
- Introducción a la regresión logística
- Regresión logística en Python
- Odds y log-odds
- **Evaluación del modelo logístico**

Evaluación del modelo logístico

Evaluación del modelo logístico

- Dado que el modelo nos proporcionó la clasificación de la probabilidad de que una persona sobreviviera al accidente del Titanic, se generó la clasificación de las probabilidades de la siguiente manera:

$$\hat{y} = \begin{cases} 1 & \text{si } p(x) \geq 0.5 \\ 0 & \text{si } p(x) < 0.5 \end{cases}$$

- En base a dicha clasificación, podemos evaluar el rendimiento del modelo.

Matriz de confusión

- La matriz de confusión es una herramienta fundamental en la evaluación del rendimiento de un modelo de clasificación.
- Proporciona una representación tabular de las predicciones realizadas por el modelo en comparación con las verdaderas etiquetas.

Matriz de confusión

La representación gráfica de la matriz corresponde a:

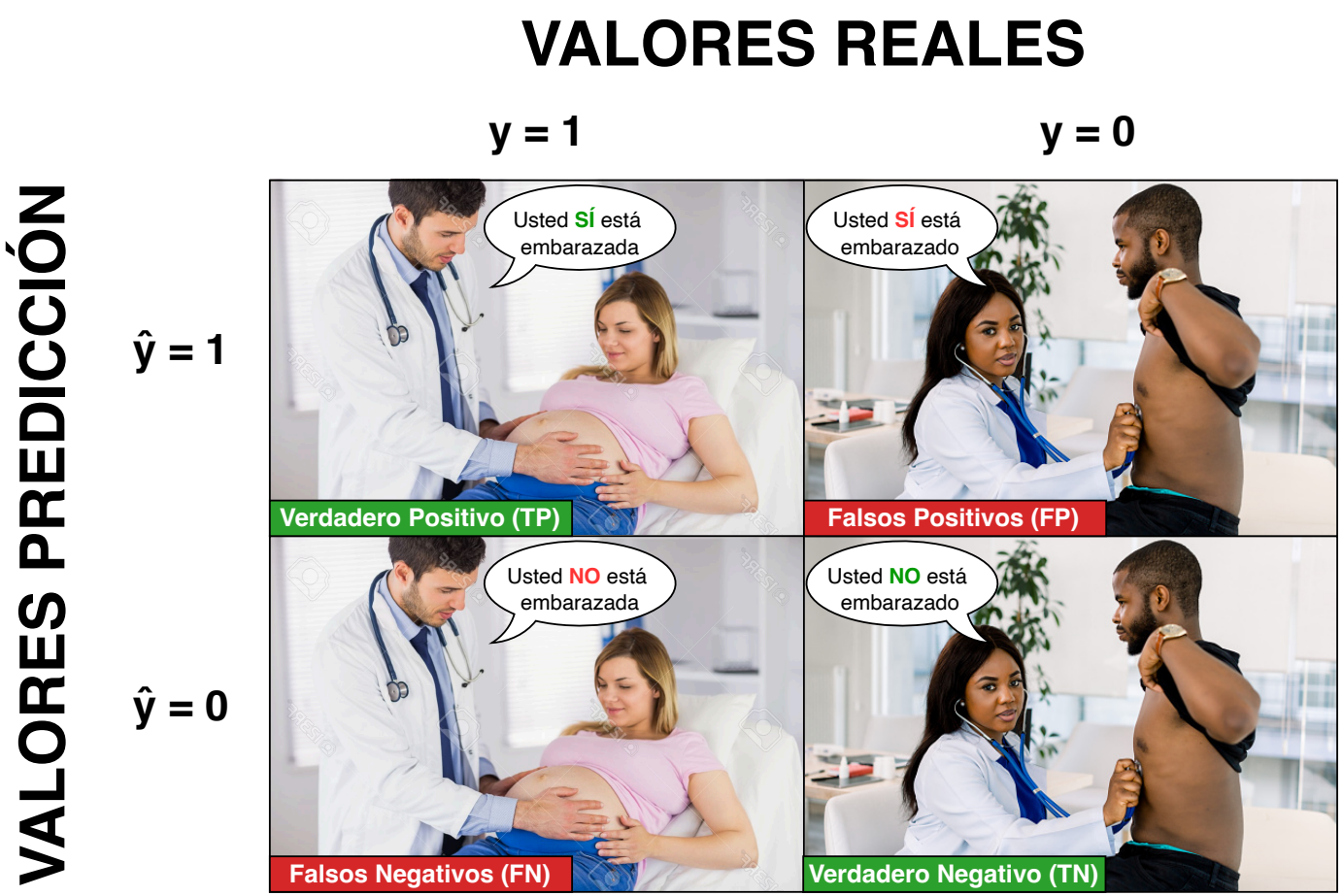
VALORES REALES

VALORES PREDICCIÓN

Verdaderos Positivos (TP)	Falsos Positivos (FP)
Falsos Negativos (FN)	Verdaderos Negativos (TN)

Matriz de confusión

O también podríamos verlo de la siguiente manera:



Matriz de confusión

La matriz está compuesta por los siguientes elementos:

- **Verdaderos Positivos (TP)**: Número de casos positivos correctamente clasificados como positivos.
- **Falsos Negativos (FN)**: Número de casos positivos incorrectamente clasificados como negativos.
- **Falsos Positivos (FP)**: Número de casos negativos incorrectamente clasificados como positivos.
- **Verdaderos Negativos (TN)**: Número de casos negativos correctamente clasificados como negativos.

Matriz de confusión en Python

```python
X_test.drop(columns=["y_prob", "odds"], inplace=True)
```

```python
from sklearn.metrics import confusion_matrix
# obtenemos la probabilidad del modelo
y_prob_pred = resultados.predict(X_test)
# generamos la clasificación según la probabilidad
y_pred = np.where(y_prob_pred >= 0.5, 1, 0)
# generamos la matriz de confusión
matriz_confusion = confusion_matrix(y_test, y_pred, labels=[1, 0])
# reportamos la matriz
print(matriz_confusion)
```

```
[[40 23]
 [ 6 74]]
```

```python
# ejemplo para validar y entender cómo se construye la matriz:
# modelo: sobrevive = 1 | realidad: sobrevive = 1
tp = np.sum(np.where((y_pred == 1) & (y_test == 1), 1, 0))
# modelo: sobrevive = 0 | realidad: sobrevive = 0
tn = np.sum(np.where((y_pred == 0) & (y_test == 0), 1, 0))
# modelo: sobrevive = 1 | realidad: sobrevive = 0
fp = np.sum(np.where((y_pred == 1) & (y_test == 0), 1, 0))
# modelo: sobrevive = 0 | realidad: sobrevive = 1
```

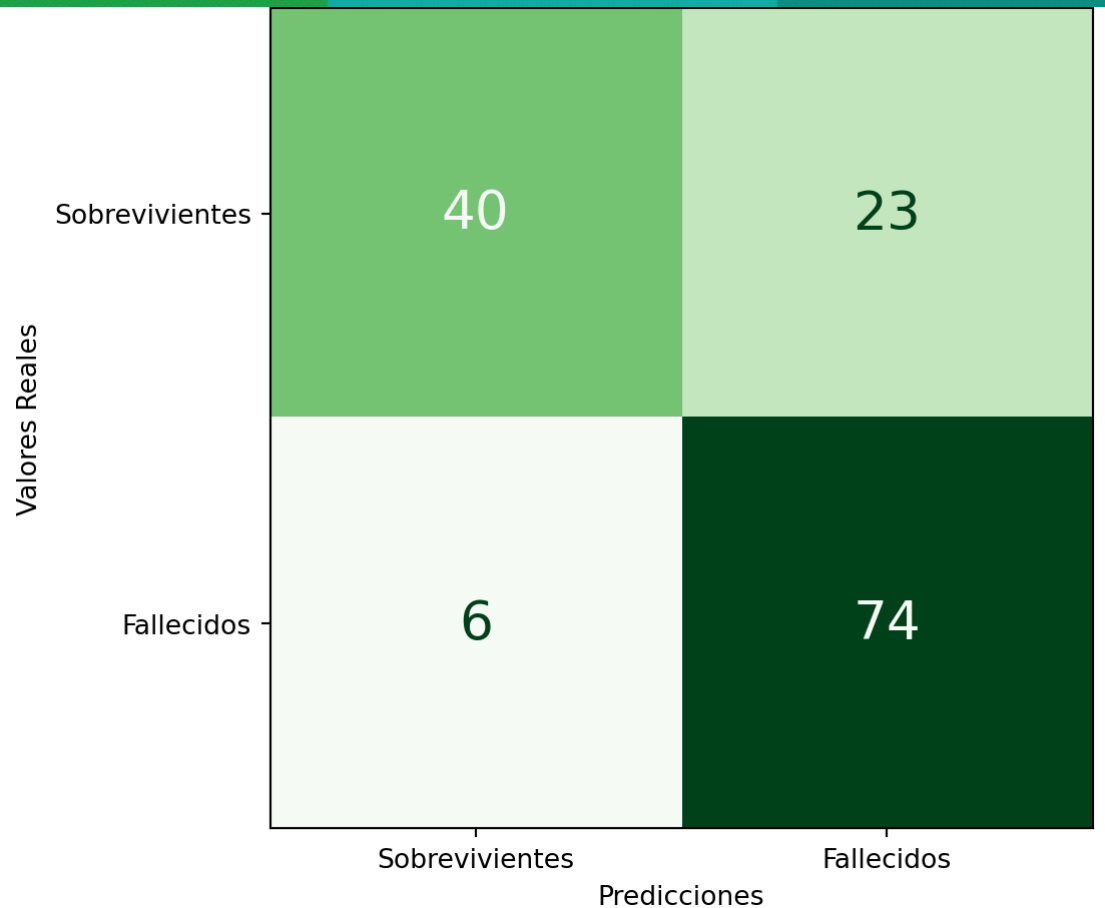
```
print(tp, tn, fp, fn)
```

```
40 74 6 23
```

Matriz de confusión en Python

O podemos usar la representación más gráfica de esta matriz:

```
from sklearn.metrics import ConfusionMatrixDisplay
fig, ax = plt.subplots()
cm = confusion_matrix(y_test, y_pred, labels=[1, 0])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Sobrevi
disp.plot(cmap="Greens", colorbar=False, ax=ax, text_kw={'size': 20})

plt.xlabel("Predicciones")
plt.ylabel("Valores Reales")

plt.tight_layout()
plt.show()
```


Sobrevivientes
Fallecidos
Valores Reales
40
23
6
74
Sobrevivientes
Fallecidos
Predicciones

Métricas de evaluación de clasificación

Métricas de evaluación de clasificación

- A partir de la matriz de confusión, podemos calcular varias métricas de evaluación importantes:
  - Precisión (Accuracy)
  - Precisión (Precision)
  - Recall (Sensibilidad)
  - F1-score

Métricas de evaluación de clasificación

Accuracy: Definición

- **Accuracy** mide la proporción de predicciones correctas (tanto verdaderos positivos como verdaderos negativos) sobre el total de predicciones realizadas.
- $\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$

Métricas de evaluación de clasificación

Accuracy: Interpretación

- **Ventajas**:
  - Es fácil de interpretar y proporciona una visión rápida de la eficacia global del modelo.
- **Desventajas**:
  - Puede resultar engañosa en conjuntos de datos desbalanceados, donde una clase es mucho más frecuente que la otra.
  - Por ejemplo, en un dataset donde el 90% de los casos son negativos, un modelo que siempre predice negativo tendrá una precisión del 90%, pero no será útil.

Métricas de evaluación de clasificación

Precision: Definición

- **Precision** es la proporción de verdaderos positivos (TP) entre todas las instancias que el modelo clasificó como positivas (TP + FP). Indica cuán precisa es la clasificación positiva del modelo.
- $\text{Precision} = \frac{TP}{TP+FP}$

Métricas de evaluación de clasificación

Precision: Interpretación

Métricas de evaluación de clasificación

Recall: Definición

- **Recall**, también conocido como sensibilidad o *True Positive Rate*, mide la proporción de verdaderos positivos entre todas las instancias que son realmente positivas (TP + FN). Indica cuán bien el modelo identifica las instancias positivas.
- $\text{Recall} = \frac{TP}{TP+FN}$

Recall: Interpretación

- **Ventajas**:
  - El recall es útil cuando las consecuencias de los falsos negativos son significativas. Por ejemplo, en un modelo de diagnóstico de enfermedades, un alto recall significa que se detectan la mayoría de los casos reales de la enfermedad.
- **Desventajas**:
  - El recall no considera los falsos positivos, por lo que un modelo puede tener un alto recall pero baja precisión.

Métricas de evaluación de clasificación

F1-score

- El F1-score es una medida equilibrada de la precisión y el recall. Combina ambas métricas en una sola, proporcionando un balance entre precisión y sensibilidad.
- $F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

Métricas de evaluación de clasificación

207 (TP)	81 (FP)
67 (FN)	357 (TN)

Accuracy

$$\frac{207 + 357}{207 + 357 + 67 + 81}$$

Precision

$$\frac{207}{207 + 81}$$

Recall

$$\frac{207}{207 + 67}$$

Desbalance de clases

Desbalance de clases

- El desbalance de clases es un problema común en el campo de machine learning, especialmente en problemas de clasificación.
- Se refiere a la situación en la que las clases del conjunto de datos no están representadas de manera equitativa.
- Esto significa que una clase tiene significativamente más instancias que la otra(s).

Desbalance de clases

Efectos en los modelos

1. **Sesgo del modelo**:

- Los modelos de clasificación pueden inclinarse hacia la clase mayoritaria, ya que están optimizados para minimizar el error general. Esto puede resultar en un modelo que predice predominantemente la clase mayoritaria, ignorando la clase minoritaria.

Desbalance de clases

Efectos en los modelos

2. **Métricas engañosas:**

- En conjuntos de datos desbalanceados, las métricas de rendimiento global como la precisión pueden ser engañosas. Un modelo que predice siempre la clase mayoritaria puede tener una alta precisión, pero su rendimiento en la clase minoritaria será deficiente.

Desbalance de clases

Ejemplo

- Imaginamos un conjunto de datos de 1000 registros para detectar fraudes financieros, donde solo 50 instancias son fraudes (1) y 950 son transacciones normales (0).
- Este conjunto de datos está desbalanceado.

```
Clase "No Fraude" (0): 950 instancias
Clase "Fraude" (1): 50 instancias
```

Manejo del Desbalance de Clases

1. **Re-muestreo**:

- **Submuestreo de la Clase Mayoritaria**: Reducir el número de instancias de la clase mayoritaria para igualar el número de instancias de la clase minoritaria.
- **Sobremuestreo de la Clase Minoritaria**: Aumentar el número de instancias de la clase minoritaria replicando instancias existentes o generando nuevas instancias sintéticas.

Manejo del Desbalance de Clases

2. **Ajuste de Ponderaciones:**

- Ajustar las ponderaciones de las clases en la función de pérdida del modelo para penalizar más los errores en la clase minoritaria.

3. **Técnicas de Algoritmos Específicos:**

- Utilizar algoritmos específicos como SMOTE (Synthetic Minority Over-sampling Technique) que generan nuevas instancias sintéticas de la clase minoritaria.